Experiment 9

Aim: To create, compile, deploy, and test an ERC-20 token smart contract using Remix IDE.

Software requirements

- Remix IDE — online Solidity development environment
- Web browser such as Chrome
- Solidity — smart contract programming language
- Remix VM — local blockchain environment for testing

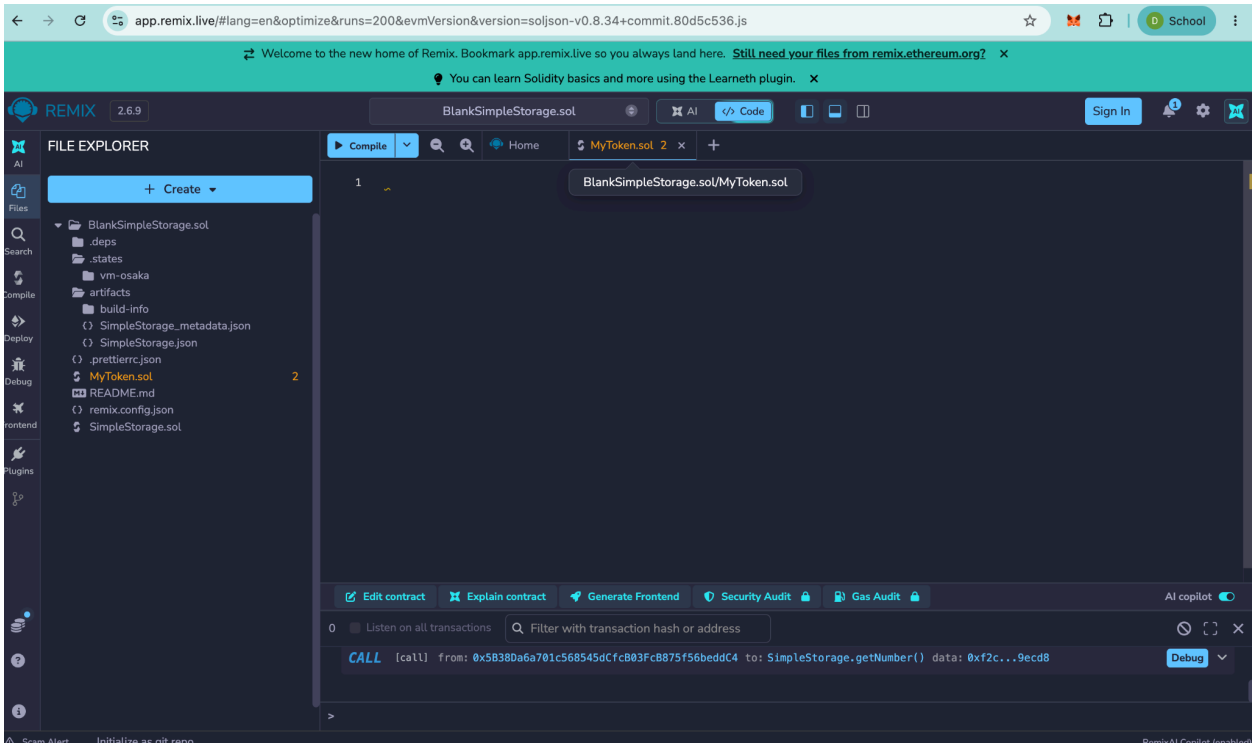
Theory: ERC-20 is a technical standard used to create fungible tokens on the Ethereum blockchain. Fungible tokens are interchangeable; for example, one token unit has the same value and properties as another unit of the same token.

An ERC-20 token contract typically supports functions such as:

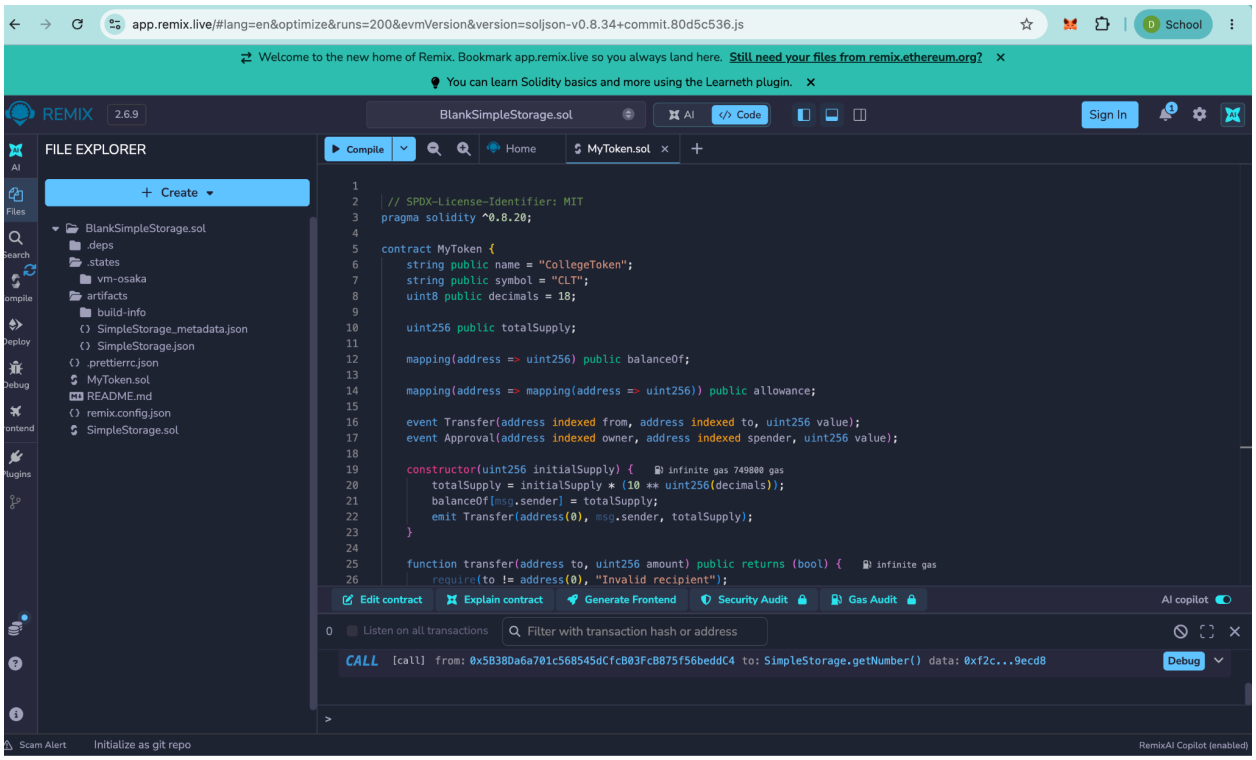
- `totalSupply()` — returns the total token supply.
- `balanceOf()` — checks the token balance of an address.
- `transfer()` — transfers tokens to another address.
- `approve()` — allows another address to spend tokens.
- `transferFrom()` — transfers tokens using an approved allowance.

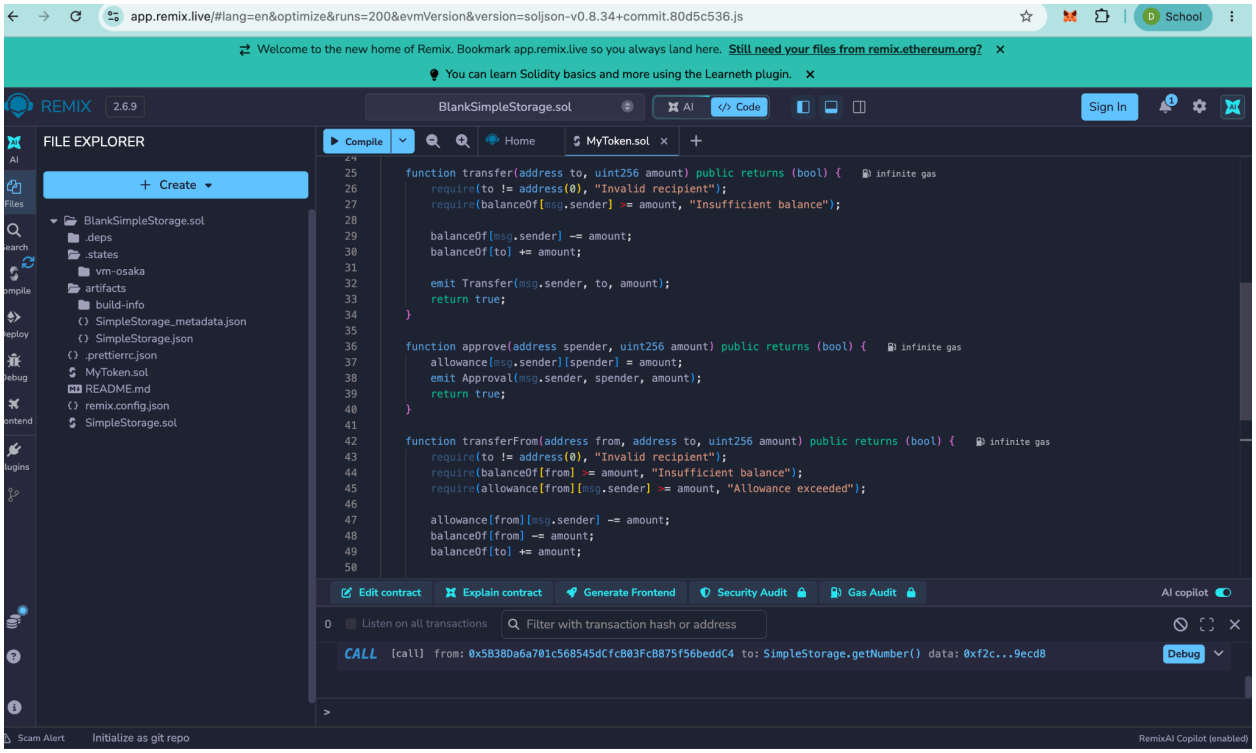
Remix IDE allows us to write Solidity code, compile it, deploy the contract to a test blockchain, and interact with its functions.

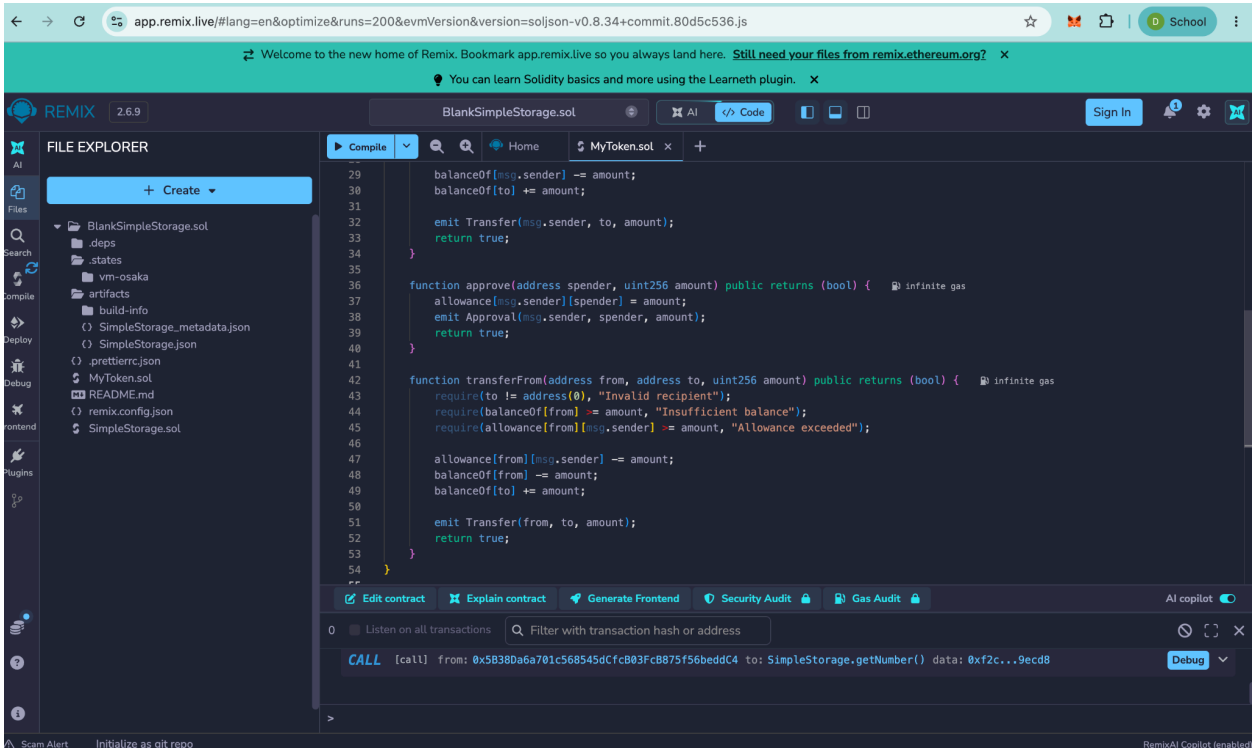
Step 1: Create a new Solidity file



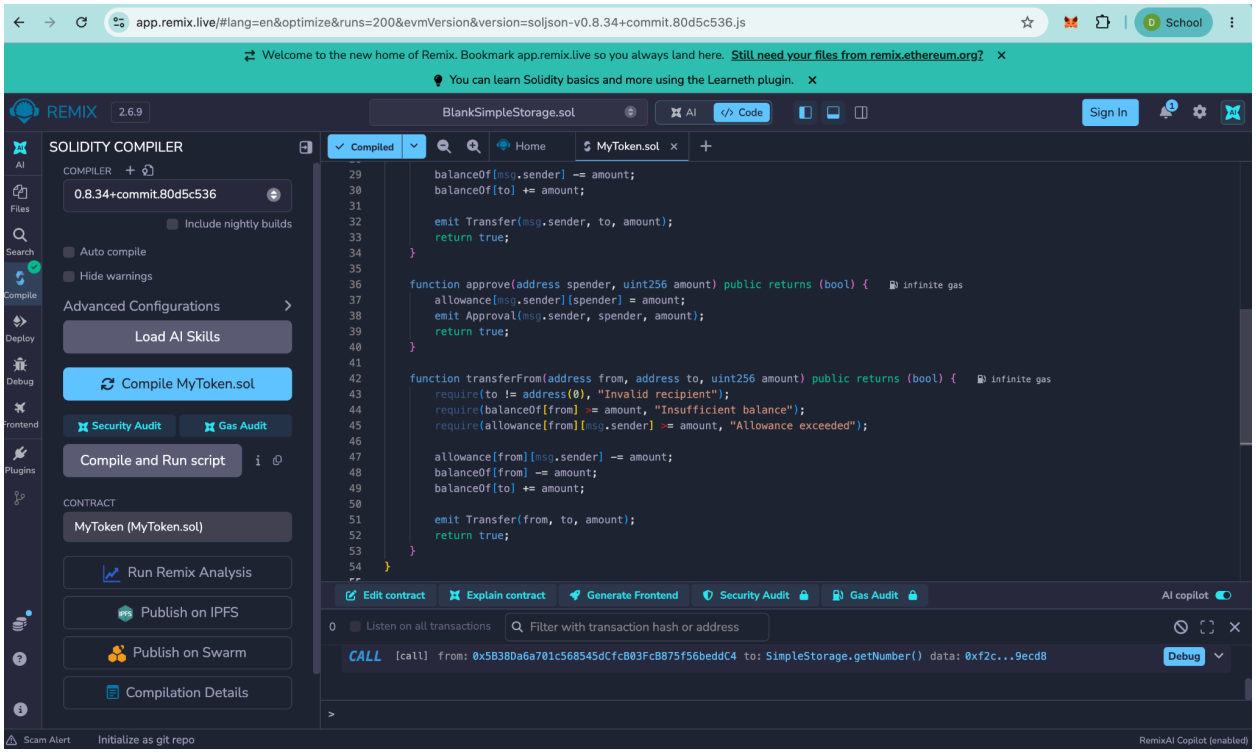
Step 2: Write the ERC-20 token program



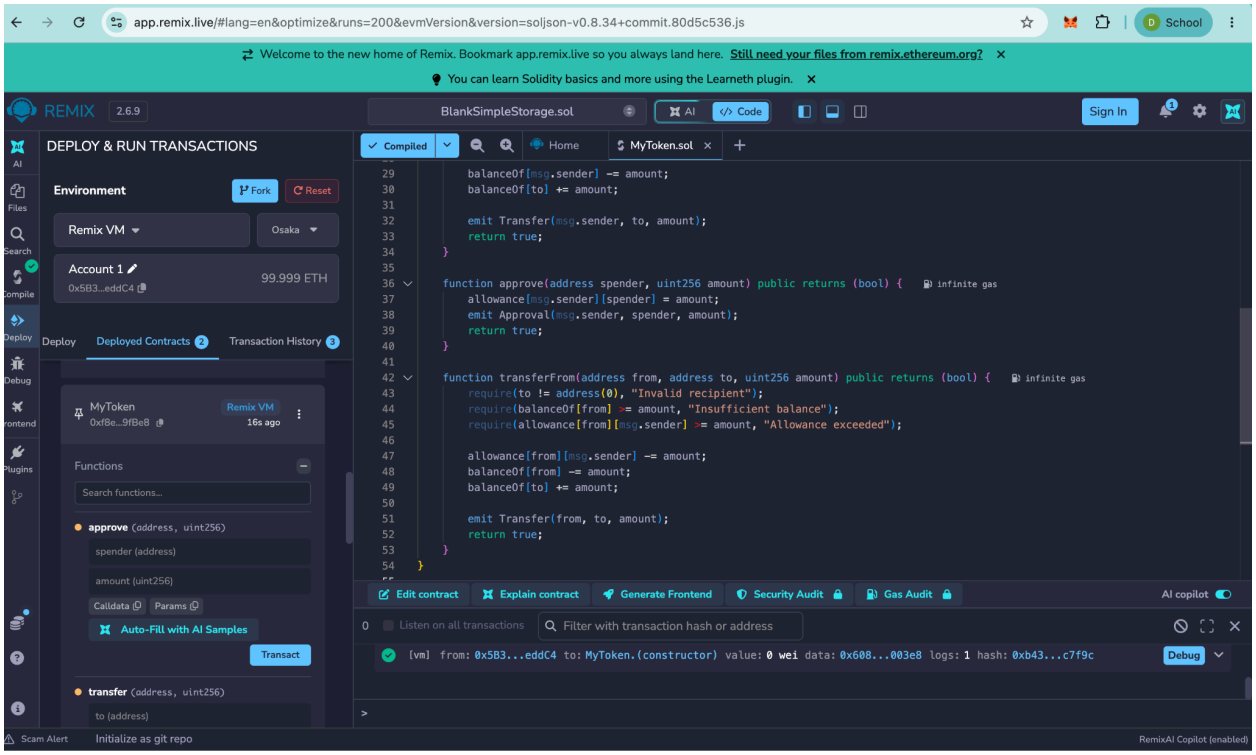

app.remix.live/#lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.34+commit.80d5c536.js
School
Welcome to the new home of Remix. Bookmark app.remix.live so you always land here. Still need your files from remix.ethereum.org?
You can learn Solidity basics and more using the Learneth plugin.
REMIX 2.6.9
BlankSimpleStorage.sol
AI
Code
Sign In
FILE EXPLORER
Create
BlankSimpleStorage.sol
.deps
.states
vm-osaka
artifacts
build-info
SimpleStorage_metadata.json
SimpleStorage.json
.prettierrc.json
MyToken.sol
README.md
remix.config.json
SimpleStorage.sol
Compile
Home
MyToken.sol
function transfer(address to, uint256 amount) public returns (bool) {
require(to != address(0), "Invalid recipient");
require(balanceOf[msg.sender] >= amount, "Insufficient balance");
balanceOf[msg.sender] -= amount;
balanceOf[to] += amount;
emit Transfer(msg.sender, to, amount);
return true;
}
function approve(address spender, uint256 amount) public returns (bool) {
allowance[msg.sender][spender] = amount;
emit Approval(msg.sender, spender, amount);
return true;
}
function transferFrom(address from, address to, uint256 amount) public returns (bool) {
require(to != address(0), "Invalid recipient");
require(balanceOf[from] >= amount, "Insufficient balance");
require(allowance[from][msg.sender] >= amount, "Allowance exceeded");
allowance[from][msg.sender] -= amount;
balanceOf[from] -= amount;
balanceOf[to] += amount;
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Listen on all transactions
Filter with transaction hash or address
CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: SimpleStorage.getNumber() data: 0xf2c...9ecd8
Debug
Scam Alert
Initialize as git repo
RemixAI Copilot (enabled)


app.remix.live/#lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.34+commit.80d5c536.js
School
Welcome to the new home of Remix. Bookmark app.remix.live so you always land here. Still need your files from remix.ethereum.org?
You can learn Solidity basics and more using the Learneth plugin.
REMIX 2.6.9
BlankSimpleStorage.sol
AI
Code
Sign In
FILE EXPLORER
Create
BlankSimpleStorage.sol
.deps
.states
vm-osaka
artifacts
build-info
SimpleStorage_metadata.json
SimpleStorage.json
.prettierrc.json
MyToken.sol
README.md
remix.config.json
SimpleStorage.sol
Compile
Home
MyToken.sol
balanceOf[msg.sender] -= amount;
balanceOf[to] += amount;
emit Transfer(msg.sender, to, amount);
return true;
}
function approve(address spender, uint256 amount) public returns (bool) {
allowance[msg.sender][spender] = amount;
emit Approval(msg.sender, spender, amount);
return true;
}
function transferFrom(address from, address to, uint256 amount) public returns (bool) {
require(to != address(0), "Invalid recipient");
require(balanceOf[from] >= amount, "Insufficient balance");
require(allowance[from][msg.sender] >= amount, "Allowance exceeded");
allowance[from][msg.sender] -= amount;
balanceOf[from] -= amount;
balanceOf[to] += amount;
emit Transfer(from, to, amount);
return true;
}
}
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Listen on all transactions
Filter with transaction hash or address
CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: SimpleStorage.getNumber() data: 0xf2c...9ecd8
Debug
Scam Alert
Initialize as git repo
RemixAI Copilot (enabled)

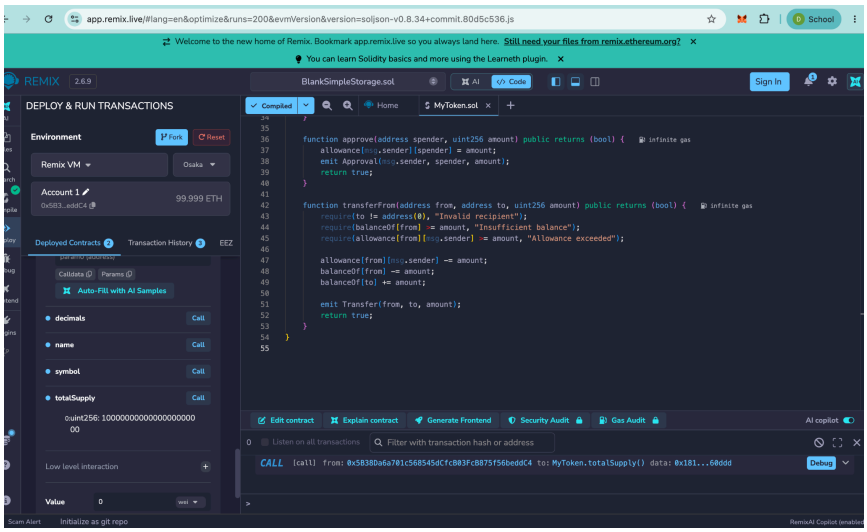
Step 3: Compile the contract

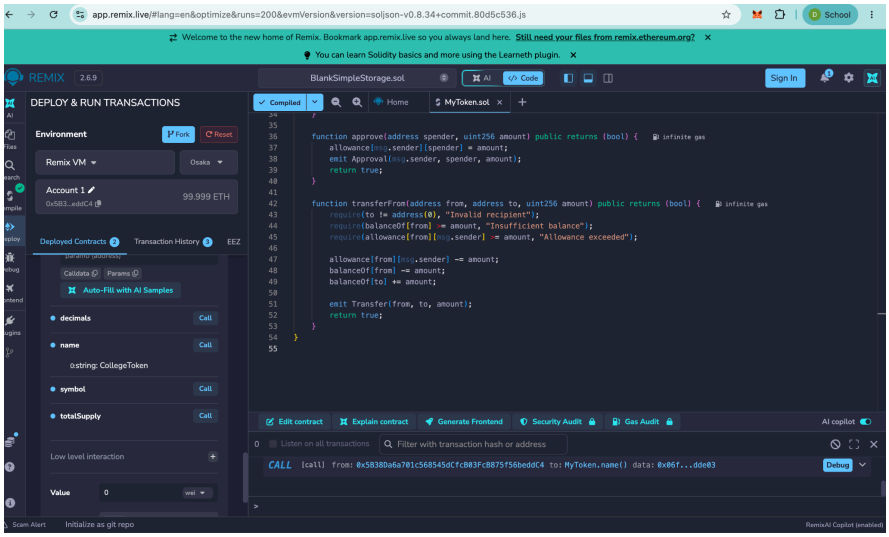


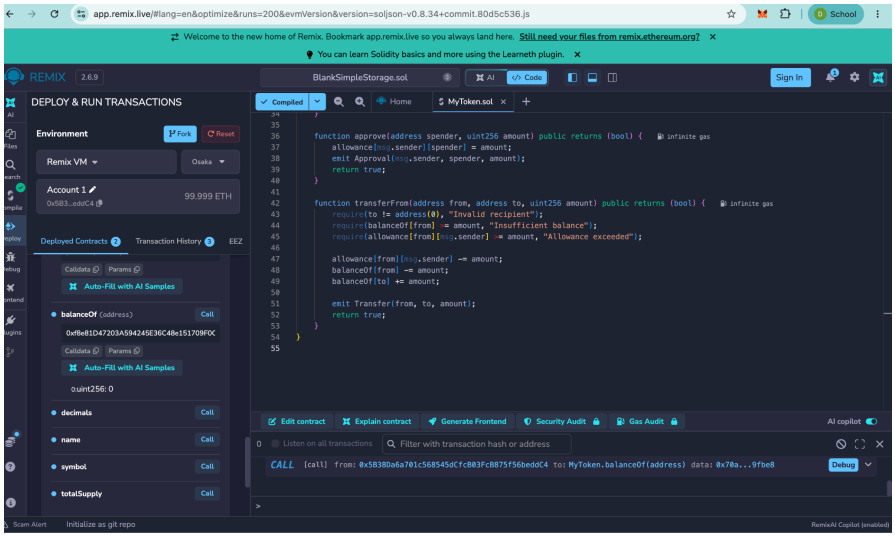
Step 4: Deploy the token



Step 5: Test the token







Conclusion:This experiment demonstrated the implementation of a token smart contract using Solidity. The token's total supply, account balances, and transfer functionality were studied and tested in the Remix VM environment.